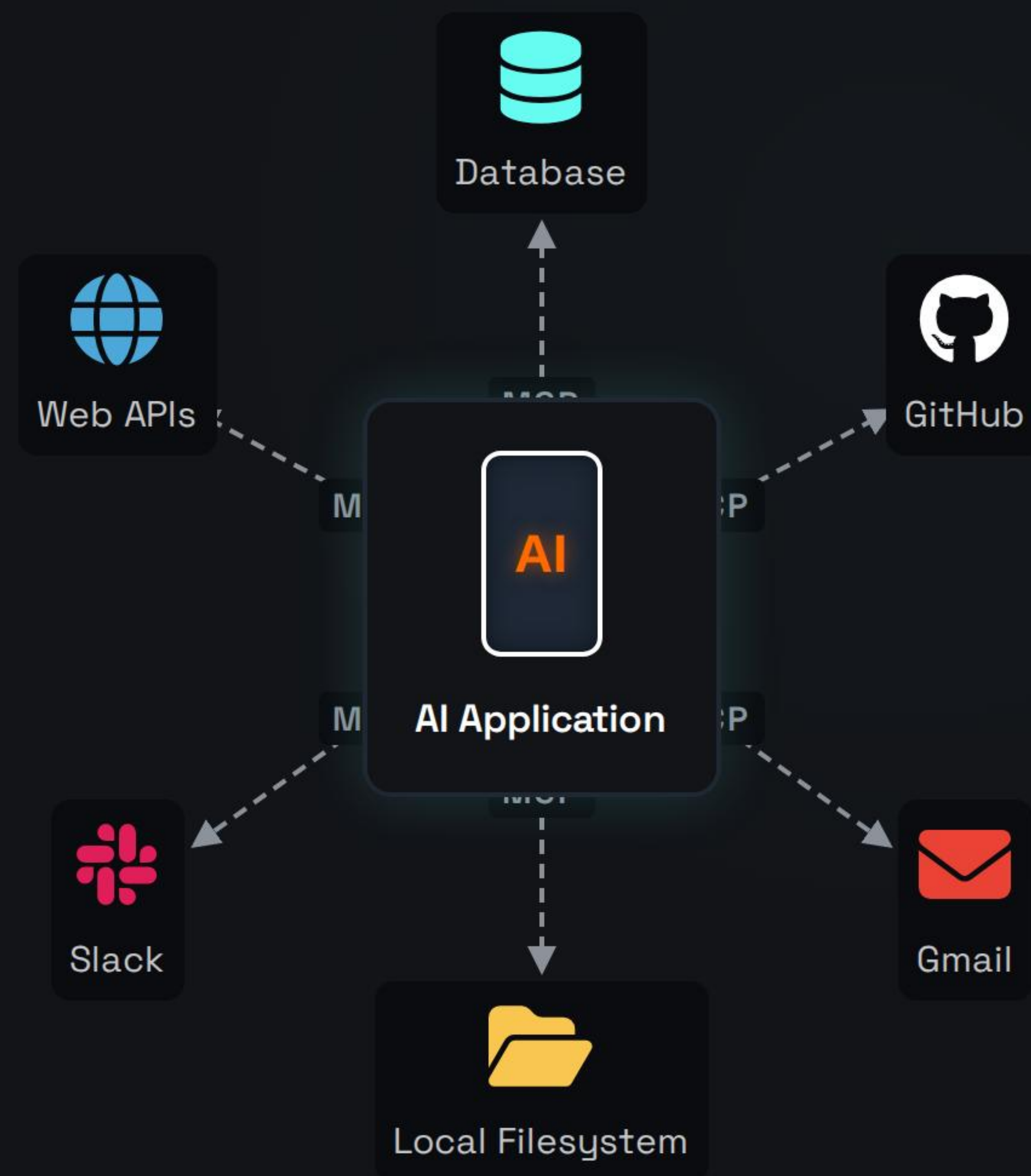


Model Context Protocol (MCP)

มาตรฐานแบบเปิด (Open Standard) ที่ช่วยให้แอปพลิเคชัน AI สามารถเชื่อมต่อกับระบบภายนอก เช่น แหล่งข้อมูลองค์กร แอปพลิเคชันทางธุรกิจ ฐานข้อมูล API และเครื่องมือ Automation ได้อย่างปลอดภัย



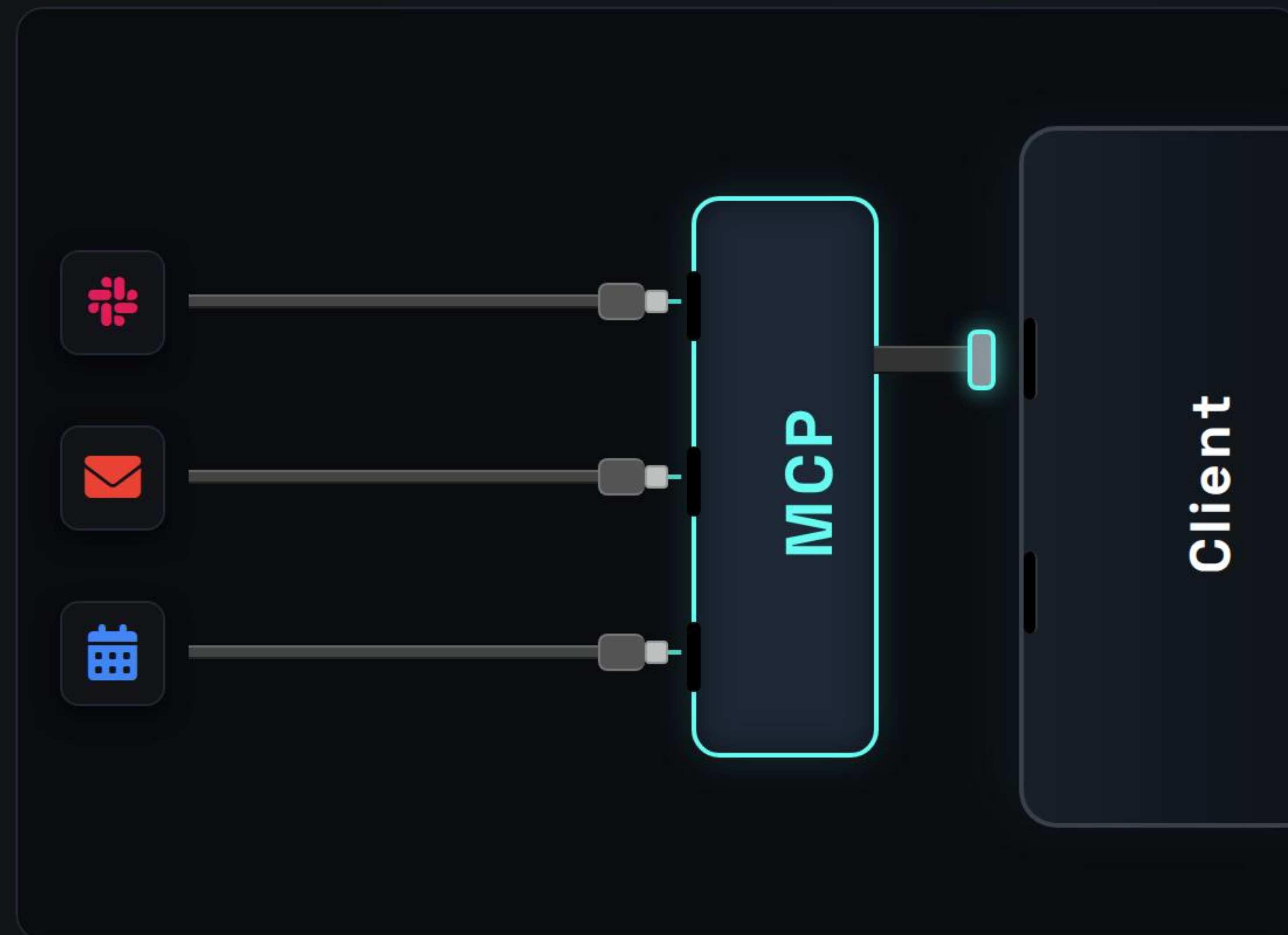
ภาพรวมของ MCP

MCP (Model Context Protocol) คือมาตรฐานแบบเปิด (Open Standard) ที่เพิ่งเปิดตัวโดย Anthropic (Nov 2024)

หน้าที่หลัก: ช่วยแก้ปัญหาที่แต่ละ AI application ต้องพัฒนา integration แบบเฉพาะเจาะจงสำหรับทุกระบบที่ต้องการเชื่อมต่อ

เปรียบเสมือน "USB-C สำหรับ AI"

ทำให้ AI client สามารถเชื่อมต่อกับบริการต่างๆ ผ่านรูปแบบการสื่อสารเดียวกัน เสียบปุ๊บ ค่อยกันรู้เรื่องปั๊บ



เหตุผลที่สร้าง MCP



ลดความซ้ำซ้อน (Build Once, Run Anywhere)

พัฒนาระบบเชื่อมต่อเพียงครั้งเดียว และนำไปใช้กับ AI Client รุ่นใดก็ได้ที่รองรับ MCP



การเข้าถึงที่มีการควบคุม (Governed Access)

ทำให้ AI assistant สามารถเข้าถึงข้อมูลและดำเนินการกับระบบธุรกิจได้อย่างมีการควบคุม



เพิ่มบริบทและประสิทธิภาพ (Context-Aware)

ทำให้ AI application มีข้อมูลบริบทที่เกี่ยวข้องมากขึ้น และสามารถช่วยทำงานจริงได้มากขึ้น

สถาปัตยกรรม (Host-Client-Server)



1. Host / Client

Host: แอปพลิเคชัน AI ตัวหลัก (เช่น Claude Desktop, ChatGPT, IDE)

Client: ระบบย่อยใน Host ที่จัดการการเชื่อมต่อกับ MCP Server และควบคุมสิทธิ์



2. MCP Server

บริการที่เปิดเผยความสามารถให้ AI application เรียกใช้งาน สื่อสารผ่าน JSON-RPC (รองรับทั้ง Local และ Remote)



3. External Systems

เป้าหมายปลายทางที่แท้จริง เช่น Enterprise API, ฐานข้อมูลองค์กร หรือระบบ Automation ที่ถูกเรียกใช้ผ่าน MCP Server

ความสามารถของ MCP Server

MCP Server สามารถเปิดเผยความสามารถหลักได้ 3 ประเภท



Tools

คืออะไร: คำสั่งหรือการดำเนินการที่ AI สามารถร้องขอได้

ตัวอย่าง: สร้าง Ticket, ค้นหาข้อมูลในฐานข้อมูล หรือเรียกใช้ Internal API



Resources

คืออะไร: ข้อมูลบริบท (Read-only)

ตัวอย่าง: เอกสาร, ไฟล์, ข้อมูลรายการ, หรือข้อมูลจากฐานข้อมูล



Prompts

คืออะไร: Template คำสั่งที่จัดเตรียมไว้

ตัวอย่าง: Prompt template หรือ workflow ที่กำหนดไว้ล่วงหน้าเพื่อช่วยให้ AI ทำงานอย่างเป็นขั้นตอน

ความต่างที่ชัดเจน: REST API vs MCP

REST API

วัตถุประสงค์หลัก: "Integration ระหว่าง Application"

ผู้ใช้งานหลัก: Application และ Service

รูปแบบ: Endpoint ที่กำหนดตายตัว

`GET /customers/123`

AI Context: ไม่ได้ออกแบบมาโดยเฉพาะ

ความสัมพันธ์: ใช้งานได้อย่างอิสระ (Core Integration)

MCP

วัตถุประสงค์หลัก: "Integration ระหว่าง AI กับระบบภายนอก"

ผู้ใช้งานหลัก: AI Host, Assistant และ Agent

รูปแบบ: Tool, Resource, Prompt ที่ค้นหาได้

`get_customer(id)`

AI Context: ออกแบบเพื่อแลกเปลี่ยน Context กับ AI

ความสัมพันธ์: มักเรียกใช้หรือครอบ REST API อยู่ภายใน

VS

สรุป: MCP ไม่ใช่สิ่งที่มาแทน REST API แต่เป็น AI Integration Layer ที่ช่วยให้ AI application ใช้งาน REST API และระบบเดิม
ได้สะดวกและปลอดภัยมากขึ้น

ควรใช้ MCP ตอนไหน?

✓ กรณีที่เหมาะสม (Good Use Cases)

- เชื่อมต่อ AI กับข้อมูลองค์กร: ดึงข้อมูลจากฐานข้อมูล, CRM, Knowledge Base
- ให้ AI ดำเนินการกับระบบ: สร้าง Ticket, ส่งข้อความ, Trigger workflow ภายใต้อัตโนมัติ
- สร้าง AI Integration ที่ใช้ซ้ำได้: ใช้ได้ทั้ง Claude, ChatGPT, IDE
- สร้าง AI Agent และ Workflow: งานที่ AI ต้องเรียกใช้หลายเครื่องมือต่อเนื่องกัน
- เข้าถึงข้อมูล Real-time: สถานะโครงการ, สินค้าคงคลัง, Operational Metrics

✗ กรณีที่ไม่จำเป็น (Not Necessary)

- เชื่อมต่อ API แบบง่ายและตายตัว: Backend คุยกับ Backend ไม่เกี่ยวข้องกับ AI
- ไม่มี AI model เกี่ยวข้อง: ไม่ต้องการให้ AI ค้นหาหรือเลือกใช้เครื่องมือ
- ข้อมูลเป็นข้อมูลคงที่ (Static): สามารถใส่ไว้ใน application หรือ prompt ได้โดยตรง

Implementation Code Examples

book-api → book-mcp-sv → client

1. Data Layer: book-api

REST API (Core Service)

ระบบ API หลังบ้านภายในองค์กร ที่จัดการ Business Logic ตั้งแต่ไม่มีส่วนเกี่ยวข้องกับ AI

IT Hardening: จำกัด Network Firewall ไม่อนุญาตให้ AI เข้าถึงตรงๆ ให้เรียกได้เฉพาะจากเครื่อง MCP Server เท่านั้น

src/app.ts (book-api)

```
import express from 'express';

const app = express();
app.use(express.json());

// Endpoint ดึงข้อมูลหลังบ้านแบบปกติ (REST)
app.get('/api/books', (req, res) => {
  const books = repository.getAll();
  res.json({ data: books });
});

app.listen(3000, () =>
  console.log('API running on port 3000')
);
```

2. Protocol Layer: book-mcp-sv

MCP Server (Integration Layer)

ตัวกลางที่แปลง REST API ให้กลายเป็น Tools ให้ AI รู้จัก และรับคำสั่งจาก AI มายิง API ต่อ

IT Hardening: เป็นจุดคัดกรอง (Sanitization Boundary) ตรวจสอบ Parameter และควบคุมสิทธิ์ก่อนส่งคำสั่งไปแตะ Core System

src/server.ts (book-mcp-sv)

```
import { Server } from "@modelcontextprotocol/sdk/server";

const server = new Server({ name: "book-mcp", version: "1.0.0" });

// ประกาศ Tool เพื่อให้ AI มองเห็นและเรียกใช้ได้
server.setRequestHandler(CallToolRequestSchema, async (req) => {
  if (req.params.name === "list_books") {
    // ติดต่อไปยัง API ภายใน
    const res = await fetch("http://api:3000/api/books");
    const data = await res.json();
    return { content: [{ type: "text", text: JSON.stringify(data) }] };
  }
});
```

3. Host Layer: client

AI Application

แอปพลิเคชันฝั่ง Client เชื่อม
ต่อผ่าน Standard I/O
(Stdio) หรือ SSE เพื่อร้องขอ
ให้ MCP Server ทำงานให้

IT Hardening: ตัว AI Model
และ Client ไม่จำเป็นต้องถือ
API Key, Secret หรือ
Password ของ Database
ภายในองค์กรเลย

index.ts (client)

```
import { Client } from "@modelcontextprotocol/sdk/client";
import { StdioClientTransport } from "@modelcontextprotocol/sdk/client/stdio";

// สร้าง Transport ซึ่งไปยัง MCP Server Process
const transport = new StdioClientTransport({
  command: "node",
  args: ["path/to/book-mcp-sv/build/index.js"]
});

const client = new Client({ name: "my-ai-app", version: "1.0.0" });
await client.connect(transport);

// สั่งทำงาน Tool ผ่าน MCP Protocol
const result = await client.callTool({ name: "list_books" });
```

Q&A

Thank you for attending the Hardening Session.